

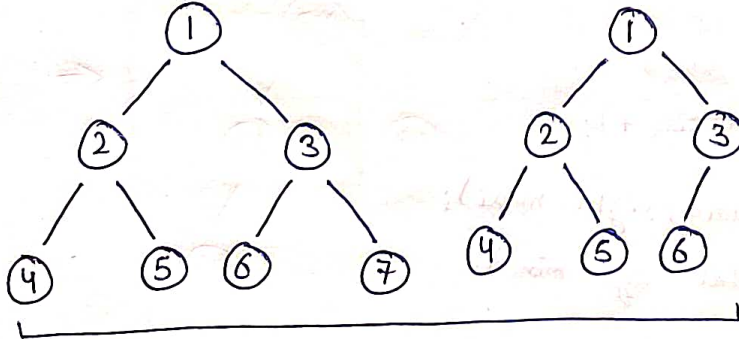
Heap

↳ is a complete Binary Tree that comes with a heap order property.

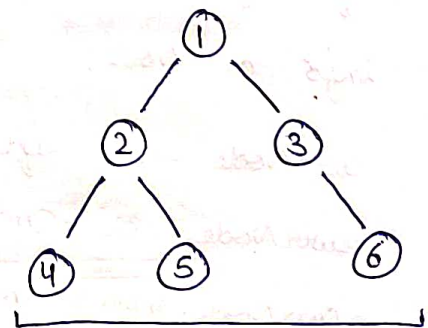
Complete Binary Tree (CBT)

↳ every level is completely filled except the last level.

↳ nodes always lean towards the left (i.e. left se nodes dalna shuru karna hai)



Complete BT

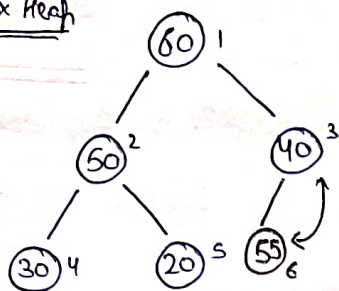


Not a CBT

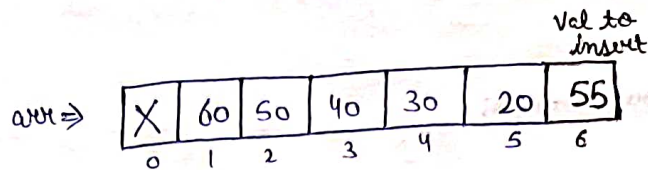
Heap Order Property → Max Heap (child ki value parent se choti Hogi)
 → Min Heap (child ki value parent se Badi Hogi)

Insertion in Heap

Max Heap



insert val = 55



Node → i^{th} index

left child → $2*i$ index

right child → $(2*i + 1)$ index

parent → $(i/2)$ index

① Insert at end

② Take it to it's current position

class heap {

public:

int arr[100];

int size = 0;

void insert (int val) {

size = size + 1;

int index = size;

arr[index] = val;

while (index > 1) {

int parent = index / 2;

if (arr[parent] < arr[index]) {

swap (arr[parent], arr[index]);

index = parent;

}

else {

return;

}

}

}

T.C. = $O(\log n)$

// Constructor

heap() {

arr[0] = -1;

size = 0;

}

```

void print() {
    for (int i=1; i <= size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

```

```

};

```

```

int main() {

```

```

    heap h;

```

```

    h.insert(50);

```

```

    h.insert(55);

```

```

    h.insert(53);

```

```

    h.insert(52);

```

```

    h.insert(54);

```

```

    h.print();

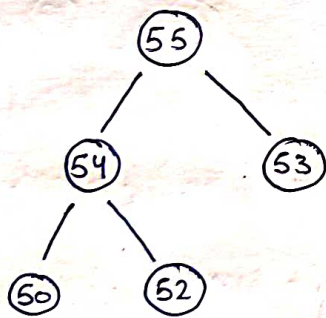
```

```

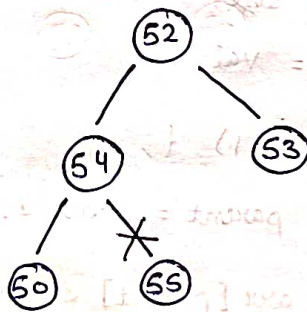
}

```

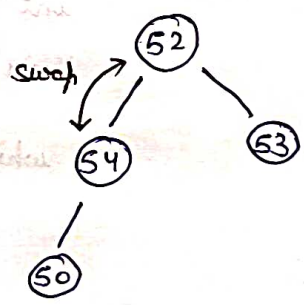
Deletion in Heap



⇒



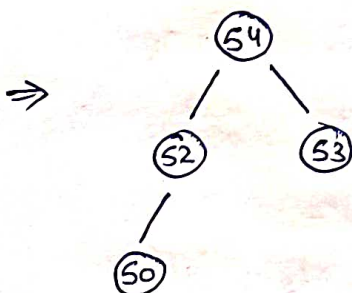
⇒



① Swap (first, last)

② Remove last node

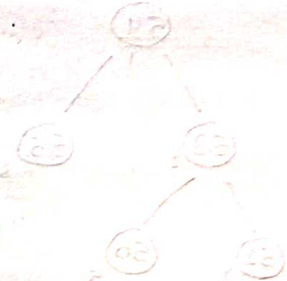
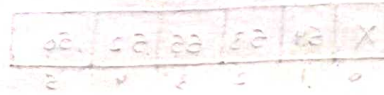
③ Propagate root node to its correct position



Delete Root

T.C. = $O(\log n)$

void deleteFromHeap() {



if (size == 0) {

cout << "nothing to delete" << endl;

return;

}

arr[1] = arr[size];

size--;

int i = 1;

while (i < size) {

int leftIndex = 2 * i;

int rightIndex = 2 * i + 1;

if (leftIndex < size && arr[i] < arr[leftIndex]) {

swap(arr[i], arr[leftIndex]);

i = leftIndex;

}

else if (rightIndex < size && arr[i] < arr[rightIndex]) {

swap(arr[i], arr[rightIndex]);

i = rightIndex;

}

else {

return;

}

}

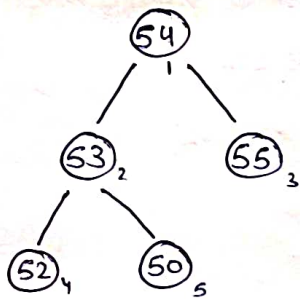
}

Heapify Algorithm

i/p $\Rightarrow$

X	54	53	55	52	50
0	1	2	3	4	5

$\Rightarrow$ Convert into Heap



X	54	53	55	52	50
0	1	2	3	4	5

←

In CBT

leaf node $\Rightarrow (\frac{n}{2} + 1)$ to n^{th} index

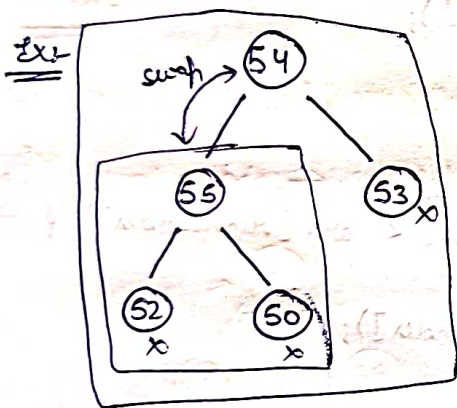
Leaf node ko process karne ki need hi nhi hai because single value hi hai compare karne ke liye kuchi hai hi nhi

so processing $\Rightarrow \frac{n}{2}$ to 0^{th} index

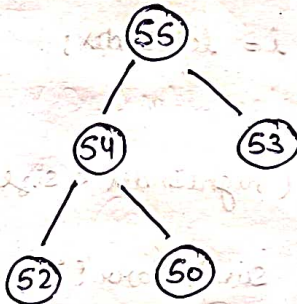
for (int i = $\frac{n}{2} \rightarrow 0$) {

 heapify(arr, size, i);

}



$\Rightarrow$



void heapify (int arr[], int n, int i) {

 int largest = i;

 int left = $2 * i$;

 int right = $2 * i + 1$;

 if (left < n && arr[largest] < arr[left]) {

 largest = left;

 }

 if (right < n && arr[largest] < arr[right]) {

 largest = right;

 }

```
if (largest != i) {
```

```
    swap(arr[largest], arr[i]);
```

```
    heapify(arr, n, largest); } O(n)
```

```
}
```

```
}
```

```
int main() {
```

```
    int arr[6] = {-1, 54, 53, 55, 52, 50};
```

```
    int n = 5;
```

```
    for (int i = n/2; i > 0; i--) {
```

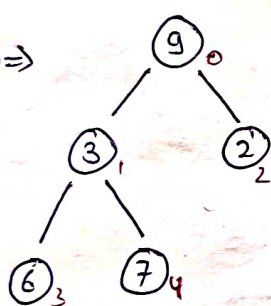
```
        heapify(arr, n, i);
```

```
    }
```

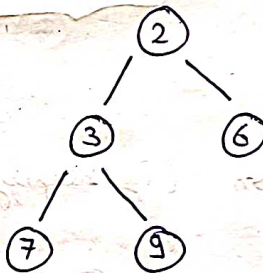
```
}
```

Qu- Build Min Heap (Input array follows 0-based indexing)

i/p $\Rightarrow$



$\Rightarrow$



```
void heapify(vector<int> &arr, int n, int i) {
```

```
    int smallest = i;
```

```
    int left = 2*i + 1;
```

```
    int right = 2*i + 2;
```

```
    if (left <= n && arr[smallest] > arr[left]) {
```

```
        smallest = left;
```

```
    }
```

```
    if (right <= n && arr[smallest] > arr[right]) {
```

```
        smallest = right;
```

```
    }
```

```
    if (smallest != i) {
```

```
        swap(arr[smallest], arr[i]);
```

```
        heapify(arr, n, smallest);
```

```
}
```

```
vector<int> buildMinHeap(vector<int> &arr) {
```

```
    int n = arr.size();
```

```
    for(int i = n/2 - 1; i >= 0; i--) {
```

```
        heapify(arr, n, i);
```

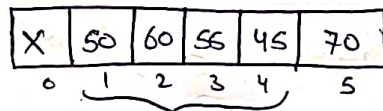
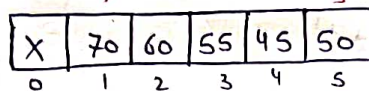
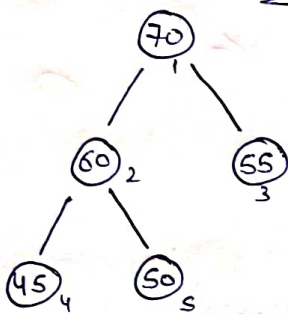
```
    }
```

```
    return arr;
```

```
}
```

Heap Sort

Max. Heap

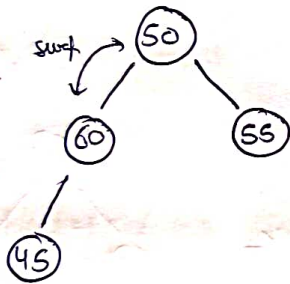


size--

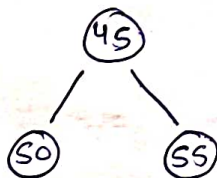
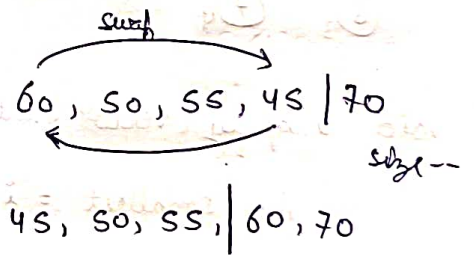
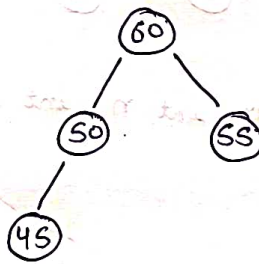
Algo:-

① swap(arr[1], arr[last])

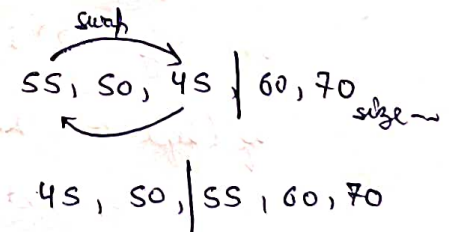
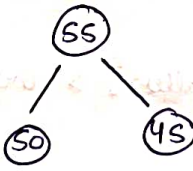
② root node → correct position (By heapify algo.)



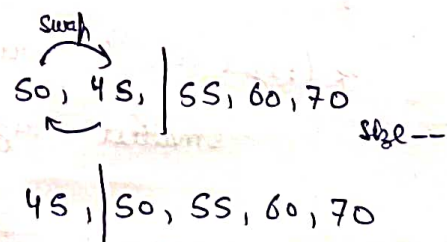
⇒



⇒



⇒



void heapSort (int arr[], int n) {

int size = n;

while (size > 1) {

swap (arr[size], arr[1]);

size--;

heapify (arr, size, 1);

}

Priority Queue

priority_queue<int> pq; // maxHeap

pq.push(4);

pq.push(2);

pq.push(5);

pq.push(3);

pq.top(); // gives 5

pq.pop();

pq.top(); // gives 4

priority_queue<int, vector<int>, greater<int>> minHeap;

Lecture 75

Ques- Kth smallest element

i/p $\Rightarrow$ {7, 10, 4, 3, 20, 15}

o/p $\Rightarrow$ 7

K = 3

Approach 1

① sort array in inc. order

T.C. = $O(n \log n)$

② return arr[K-1]

Approach ②

→ Create a maxHeap for first K element

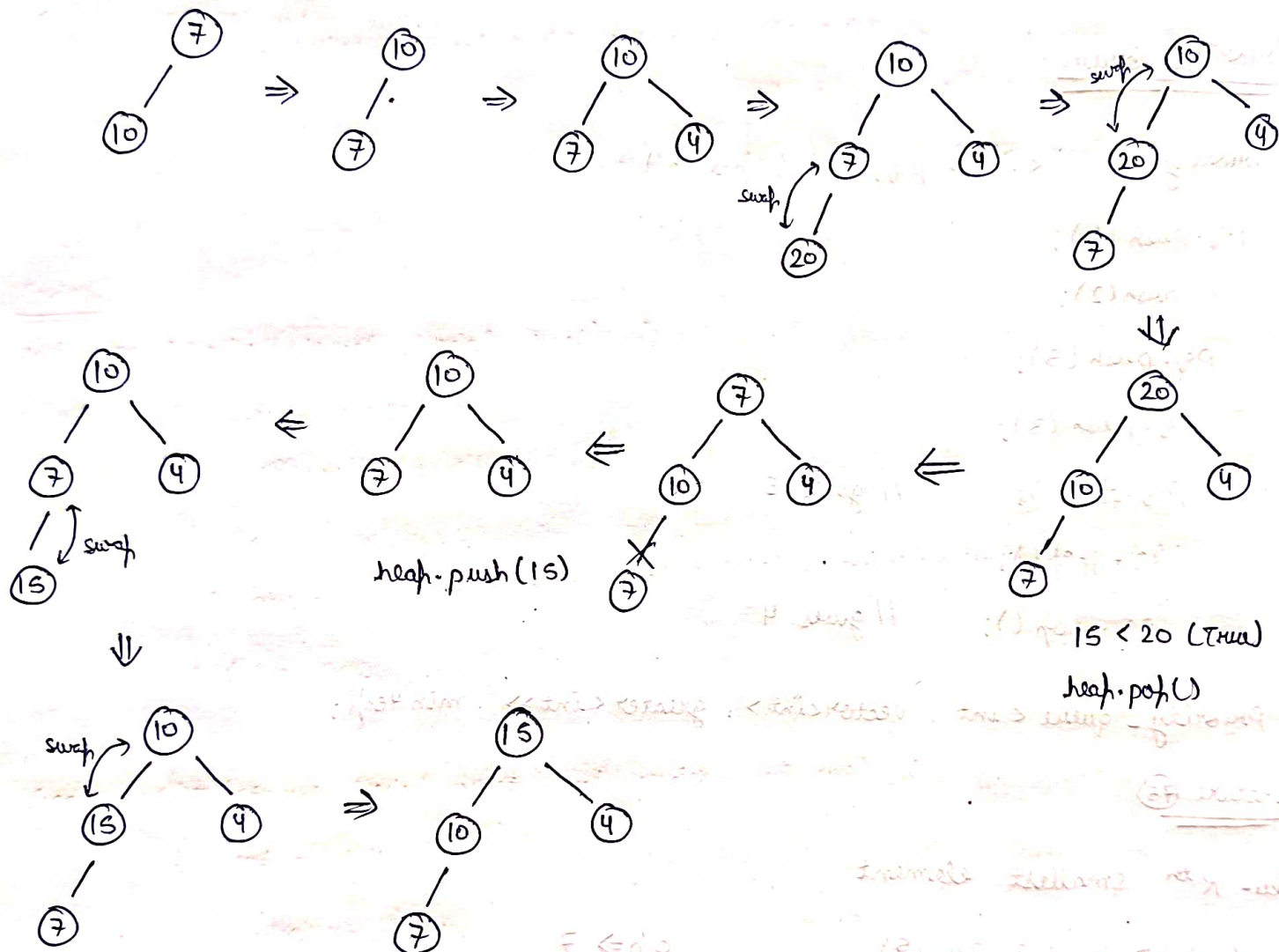
→ For rest elements, if element < heap.top()

→ heap.pop()

→ heap.push(element)

i/p $\Rightarrow \{7, 10, 4, 20, 15\}$
maxHeap

K=4



```
int KthSmallest (int arr[], int l, int r, int K) {
```

```
    priority_queue<int> pq;
```

```
    for (int i=0; i<K; i++) {
```

```
        pq.push(arr[i]);
```

```
    }
```

```
    for (int i=K; i<=r; i++) {
```

```

    if (arr[i] < pq.top()) {
        pq.pop();
        pq.push(arr[i]);
    }
}

int ans = pq.top();

return ans;
}

```

Same way me agar Hamne Kth largest pocha Hoga to min Heap se nikalenge

Qn- Check whether the given Binary Tree is max Heap

Algo:-

```

solve() {

```

```

    if (isCBT() && isMaxOrder()) {

```

```

        return true;
    }

```

```

}

```

```

else {

```

```

    return false;
}

```

```

}

```

```

isCBT (root, i, nodeCount) {

```

```

    if (root == NULL) {

```

```

        return true;
    }

```

```

}

```

```

    if (i >= nodeCount) {

```

```

        return false;
    }

```

```

}

```

```

else {

```

```

    left = isCBT(root->left, 2i+1, nodeCount);

```

```

    right = isCBT(root->right, 2i+2, nodeCount);

```

```

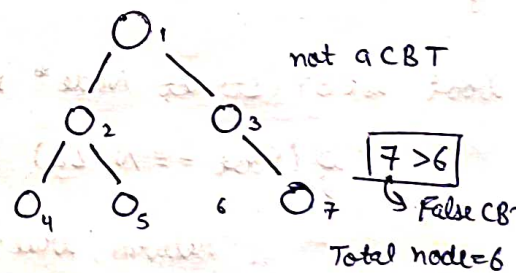
    return (left && right);
}

```

```

}

```



} For 0-based indexing

isMaxOrder(root) {

if (leaf node) {

return true;

}

if (right == NULL) {

return root->data > root->left->data;

}

else {

return root->data > root->left->data &&

root->data > root->right->data &&

isMaxOrder(left) && isMaxOrder(right);

}

}

Code:-

int countNodes(struct Node* root) {

if (root == NULL) {

return 0;

}

int ans = 1 + countNodes(root->left) + countNodes(root->right);

return ans;

}

bool isCBT(struct Node* root, int index, int cnt) {

if (root == NULL) {

return true;

}

if (index >= cnt) {

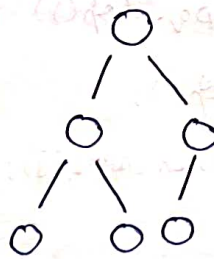
return false;

}

else {

bool left = isCBT(root->left, 2*index+1, cnt);

bool right = isCBT(root->right, 2*index+2, cnt);



(i) Both child exist

(ii) leaf node

(iii) Only left child exist

return (left && right);

}

}

bool isMaxOrder (struct Node* root) {

if (root->left == NULL && root->right == NULL) {

return true;

}

if (root->right == NULL) {

return (root->data > root->left->data);

}

else {

bool left = isMaxOrder (root->left);

bool right = isMaxOrder (root->right);

if (left && right && (root->data > root->left->data &&
root->data > root->right->data) {

return true;

}

else {

return false;

}

}

}

bool isHeap (struct Node* root) {

int index = 0;

int totalCount = countNodes (root);

if (isCBT (root, index, totalCount) && isMaxOrder (root)) {

return true;

}

else {

return false;

}

}

T.C. = $O(n)$

Qn- Merge two Binary Max-Heap

Algo:-

① Merge two vectors in a new vector

② Make Heap from that new vector by Heapify algo.

```
vector<int> mergeHeaps(vector<int> &a, vector<int> &b, int n, int m) {  
    vector<int> ans;
```

```
    for(auto i:a) {
```

```
        ans.push-back(i);
```

```
    }
```

```
    for(auto i:b) {
```

```
        ans.push-back(i);
```

```
    }
```

```
    int d = ans.size();
```

```
    for (int i = d/2-1; i >= 0; i--) {
```

```
        heapify(ans, n, i);
```

```
    }
```

```
    return ans;
```

```
}
```

} 0-based indexing

Qn- Minimum Cost of Mops

i/p $\Rightarrow$ {4, 3, 2, 6}

o/p $\Rightarrow$ 29

Explanation:-

2 + 3 = {4, 5, 6}
 /
 5

4 5 = {9, 6}
 /
 9

9 6 = {15}
 /
 15

ans = 5 + 9 + 15 = 29

```

long long minCost (long long arr[], long long n) {
    priority_queue<long long, vector<long long>, greater<long long>> pq;
    for (int i=0; i<n; i++) {
        pq.push(arr[i]);
    }
    int cost = 0;
    while (pq.size() > 1) {
        long long a = pq.top();
        pq.pop();
        long long b = pq.top();
        pq.pop();
        long long sum = a + b;
        cost += sum;
        pq.push(sum);
    }
    return cost;
}

```

T.C. = $O(n \log n)$
S.C. = $O(n)$

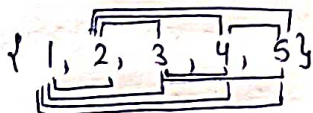
H.W.
 Qu- Convert BST to Min Heap

Qu- Kth largest sum subarray

i/p $\Rightarrow \{1, 2, 6, 4, 3\}$

o/p $\Rightarrow$ Kth largest subarray sum

Approach 1:-



```

① for (i=0  $\rightarrow$  n) {
    for (j=i  $\rightarrow$  n) {
        all subarray find
    }
}

```

- ② sum store in vector
- ③ sort in inc. order
- ④ Return Kth largest

~~T.C. = $O(n^2 \log n)$~~
T.C. = $O(n^2 \log n)$

```

int getKthLargest(vector<int> &arr, int k) {
    vector<int> sumStore;

    int n = arr.size();

    for (int i = 0; i < n; i++) {
        int sum = 0;

        for (int j = i; j < n; j++) {
            sum += arr[j];
            sumStore.push_back(sum);
        }
    }

    sort(sumStore.begin(), sumStore.end());

    return sumStore[sumStore.size() - k];
}

```

S.C. = $O(n^2)$

Approach ②:-

```

for (i = 0 → n) {
    for (j = i → n) {
        sum
        if (minHeap.size() < k) {
            insert sum
        }
        else {
            if (sum > minHeap) {
                minHeap.pop();
                push(sum);
            }
        }
    }
}

```

```
int getKthLargest(vector<int> &arr, int k) {
```

```
priority_queue<int, vector<int>, greater<int>> min;
```

```
int n = arr.size();
```

S.C. = $O(K)$

```
for (int i=0; i<n; i++) {
```

```
    int sum = 0;
```

```
    for (int j=i; j<n; j++) {
```

```
        sum += arr[j];
```

```
        if (min.size() < k) {
```

```
            min.push(sum);
```

```
        }
```

```
    } else {
```

```
        if (sum > min.top()) {
```

```
            min.pop();
```

```
            min.push(sum);
```

```
        }
```

```
    }
```

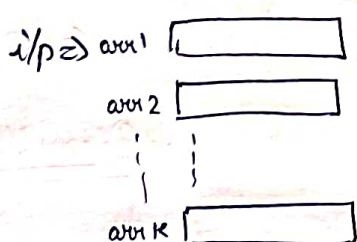
```
}
```

```
}
```

```
return min.top();
```

```
}
```

Qn- Merge K sorted arrays



} K arrays

merge $\Rightarrow$



Approach ①:-

- ① Insert all element in new array
- ② sort an array

} T.C. = $O(nK \log(nK))$

Approach ②

minHeap $\rightarrow$ while (minHeap.size() > 0) {
starting me Har array ka first element Hoga

minHeap $\rightarrow$ ans array

& insert next element of same array into heap if present

}

$$T.C. = O(nK \log K)$$

```
class node {
```

```
public:
```

```
    int data;
```

```
    int i;
```

```
    int j;
```

```
};
```

```
// Comparator
```

```
node (int data, int row, int col) {
```

```
    this->data = data;
```

```
    i = row;
```

```
    j = col;
```

```
}
```

```
class compare {
```

```
public:
```

```
    bool operator()(node* a, node* b) {
```

```
        return a->data > b->data;
```

```
    }
```

```
};
```

```
vector<int> mergeKSortedArrays (vector<vector<int>> & KArrays, int K) {
```

```
    priority_queue<node*, vector<node*>, compare> minHeap;
```

```
    for (int i=0; i<K; i++) {
```

```
        node* tmp = new node (KArraysarr[i][0], i, 0);
```

```
        minHeap.push(tmp);
```

```
    }
```

```
    vector<int> ans;
```

```
    while (minHeap.size() > 0) {
```

```
        node* tmp = minHeap.top();
```

```
        ans.push_back(minHeap.top().data);
```

```
        minHeap.pop();
```

```
        int i = tmp->i;
```

```
        int j = tmp->j;
```

```
if (j+1 < KArrays[i].size()) {
```

```
node* next = new node(arrKArrays[i][j+1], i, j+1);
```

```
minHeap.push(next);
```

```
}
```

```
}
```

```
return ans;
```

```
}
```

Qn- Merge K sorted Linked List

Approach ①:-

① Make vector<int> ans & store all elements of all linked list

② sort the vector

T.C = $O(nK \log(nK))$

③ Make L.L. from that vector

Approach ②:-

minHeap $\rightarrow$ first element of K linked list

```
while (!minHeap.empty()) {
```

```
if (head == NULL) {
```

```
head = tail = minHeap.top();
```

```
minHeap.pop();
```

```
if (head->next != NULL) {
```

```
minHeap.insert
```

```
}
```

```
else {
```

```
tail->next = minHeap.top();
```

```
minHeap.pop();
```

```
tail = tail->next;
```

```
if (tail->next != NULL) {
```

```
minHeap.insert
```

```
}
```

```
}
```

```
}
```

class compare {

public:

bool operator() (Node<int>* a, Node<int>* b) {

return a->data > b->data;

}

};

Node<int>* mergeKLists (vector<Node<int>*> &listArray) {

priority_queue<Node<int>*, vector<Node<int>*>, compare> minHeap;

int K = listArray.size();

if (K == 0) {

return NULL;

}

for (int i = 0; i < K; i++) {

if (listArray[i] != NULL) {

minHeap.push(listArray[i]);

}

}

Node<int>* head, tail = NULL;

while (minHeap.size() > 0) {

Node<int>* temp = minHeap.top();

minHeap.pop();

if (head == NULL) {

head = temp;

tail = temp;

if (tail->next != NULL) {

minHeap.push(tail->next);

}

}

else {

T.C = $O(nK \log K)$

S.C = $O(K)$

tail → next = temp;

tail = temp;

if (tail → next != NULL) {

minHeap.push(tail → next);

}

}

}

return head;

}

Qu- Smallest Range in K-List

i/p ⇒ $\begin{cases} \square \rightarrow \square \rightarrow \square \rightarrow x \\ \square \rightarrow \square \rightarrow \square \rightarrow x \\ \vdots \\ \square \rightarrow \square \rightarrow \square \rightarrow x \end{cases}$

o/p ⇒ $[x, y]$

Algo smallest range jiske andar Har list ka atleast 1 element aa jaye

Approach ①:-

K=3 {1, 10, 11}

{2, 3, 20}

{5, 6, 12}

① Saari Range nikal lo (Total elements = $n \times k$)

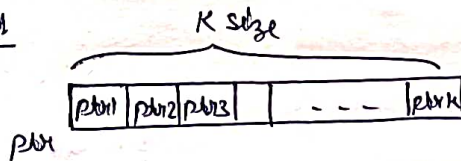
② Uska difference store karo aur smallest return karo with atleast 1 element from each list

T.C = $O(n^2 k^2)$

Approach ②:-

$\begin{cases} \text{ptr1} \rightarrow \{1, 10, 11\} \\ \text{ptr2} \rightarrow \{2, 3, 20\} \\ \text{ptr3} \rightarrow \{5, 6, 12\} \end{cases}$

K-pointers



min/max

min = 1, max = 5

[1, 5]

satisfied

Smallest Range & (atleast 1 element)

(i) min ko Badha do

ya

(ii) max ko ghata do

X Because list is sorted

so, only one option min ko Badha do

Now, 10, 2, 5

min = 2
max = 10

[2, 10] → diff. → if smallest then store

$$T.C. = O(nk^2)$$

$$S.C. = O(k)$$

Approach ③:-

{1, 10, 11}

{2, 3, 20}

{5, 6, 12}

- ① minHeap of K size jisme Har list Ka starting element dal do

② delete hme apne max track krna hai

H.W.

① Code the above problem

② Ques- Median in a Stream

③ Ques- Reorganize strings

Hashmaps

→ Insertion, deletion, searching $\Rightarrow O(1)$

Ques- string str = "abcdeaaaf", find maximum occurring character

a → 3

b → 2

c → 1

d → 1

e → 1

f → 1

o/p $\Rightarrow$ 3 times i.e. a

Algo:-

→ a → count → store in ans.
loop from a to z

Approach ②:-

count =

0	1	2	3	...	25
---	---	---	---	-----	----

↓ ↓ ↓ ↓
'a' 'b' 'c' 'z'

```
for (i = 0 → str.size()) {  
    char ch = str[i];  
    count[ch - 'a']++;  
}
```

Then
display
max. count

To ask question me to Ham aaram se solve kar sakte hai but
let's say the string is like :-

str = "mera naam hai babbar" now max. occurring word
mera babbar babbar"

Solⁿ

mera $\rightarrow 2$

naam $\rightarrow 1$

hai $\rightarrow 1$

babbar $\rightarrow 3$

Max. so, answer o/p $\Rightarrow$ babbar

To agar poor words ki mapping karni hai jaise $\langle \text{mera}, 2 \rangle$ then
 $\langle \text{naam}, 1 \rangle$
 $\langle \text{babbar}, 3 \rangle$
we need hashmap

class node {

$\xRightarrow{\text{in map}}$

map < node, int > m;

};

Implementation

$\hookrightarrow$ Using Linked List (Insertion, deletion & searching in $O(n)$ not in $O(1)$)

$\hookrightarrow$ Using BST (Again above 3 is in $O(\log n)$ not in $O(1)$)

So, we will use Hash Table

Inbuilt stuff

$\hookrightarrow$ map $\Rightarrow O(\log n)$

#include <map>

$\hookrightarrow$ unordered-map $\Rightarrow O(1)$

#include <unordered-map>

int main() {

// creation

unordered-map < string, int > m;

// insertion

pair < string, int > p = make_pair("babbar", 3);

m.insert(p);

} 1st way

```
pair<string, int> pair2("love", 2);
```

```
m.insert(pair2);
```

} 2nd way

```
m["mera"] = 1;
```

} 3rd way

```
// search
```

```
cout << m["mera"] << endl;      1
```

```
cout << m.at("babbar") << endl;      3
```

```
cout << m.at("unknownkey") << endl;      out of range
```

```
cout << m["unknownkey"] << endl;      0
```

```
cout << m.at("unknownkey") << endl;      0
```

```
cout << m.size() << endl;      4
```

```
// to check presence
```

```
cout << m.count("pro") << endl;      0
```

```
cout << m.count("love") << endl;      1
```

} check presence

```
// erase
```

```
m.erase("love");
```

```
cout << m.size() << endl;      3
```

```
for (auto i : m) {
```

```
    cout << i.first << " " << i.second << endl;
```

```
}
```

```
// iterator
```

```
unordered_map<string, int> :: iterator it = m.begin();
```

```
while (it != m.end()) {
```

```
    cout << it->first << " " << it->second << endl;
```

```
    it++;
```

```
}
```

⇒ Unordered-map ⇒ Prints in random order

map ⇒ Prints in insertion sequence

Ques- Maximum frequency number

```
int maximumFrequency (vector<int> &arr, int n) {
```

```
    unordered_map<int, int> count;
```

```
    for (int i=0; i<arr.size(); i++) {
```

```
        count[arr[i]]++;
```

```
    }
```

```
    int maxi = INT_MIN;
```

```
    int ans = -1; count
```

```
    for (auto i: count) {
```

```
        if (i.second > maxi) {
```

```
            maxi = i.second;
```

```
            ans = i.first;
```

```
        }
```

```
    }
```

```
}
```

This code gives wrong answer

```
int maximumFrequency (vector<int> &arr, int n) {
```

```
    unordered_map<int, int> count;
```

```
    int maxFreq = 0;
```

```
    int maxAns = 0;
```

```
    for (int i=0; i<arr.size(); i++) {
```

```
        count[arr[i]]++;
```

```
        maxFreq = max(maxFreq, count[arr[i]]);
```

```
    }
```

```
    for (int i=0; i<arr.size(); i++) {
```

```
        if (maxFreq == count[arr[i]]) {
```

```
            maxAns = arr[i];
```

```
            break;
```

```
        }
```

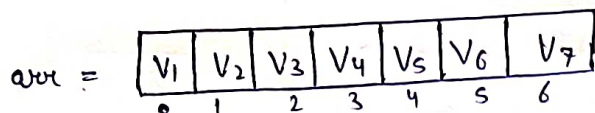
```
    }
```

```
    return maxAns;
```

```
}
```

Bucket Array

< key, value >



Hash functions

↳ combination of Hash code and compression function

String
char
object } $\xrightarrow[\text{Hash code}]{\text{conversion is done by}}$ int

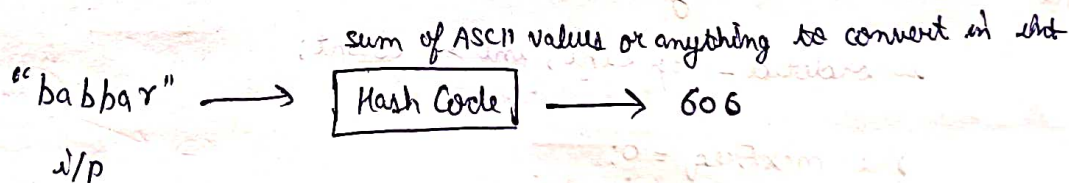
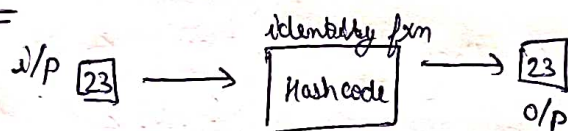
} for mapping & storing
values in Bucket array

Keys ko Bucket array ke size ki range me lane ka kaam compression fn.

int from Hash code $\xrightarrow[\text{compression function}]{\text{compression}}$ Range

Hash Code

identity fn



Why Hash Code? $\rightarrow$ conversion to int
 $\rightarrow$ uniform distribution

Collision

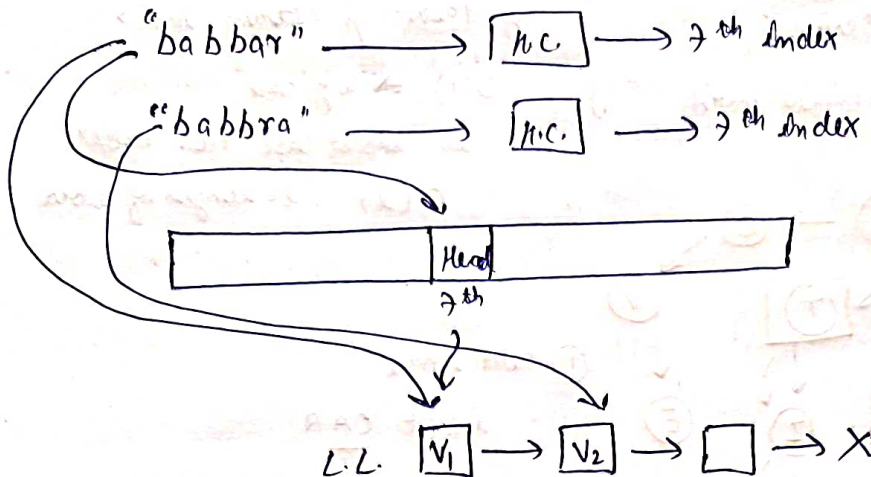


so, we need to do collision Handling

Collision Handling

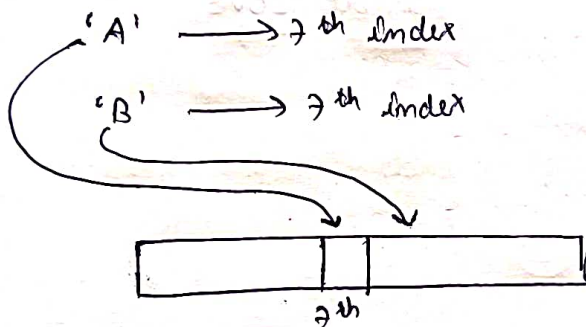
siml

① Open Hashing $\Rightarrow$ Same place par hi jao



separate chaining

② Open Addressing $\Rightarrow$ Agli jagah par daal do



Agli jagah depend karegi function par:-

$$H_i(a) = h(a) + F_i(a) \quad \left. \begin{array}{l} \text{ } \\ \text{ } \end{array} \right\} \begin{array}{l} \text{ith attempt me kaha par place} \\ \text{karna hai} \end{array}$$

$\Rightarrow$ Linear Probing :- $F(i) = i$

$\Rightarrow$ Quadratic Probing :- $F(i) = i^2$

Complexity Analysis

"mera bhai love babbar bro kya haal chaal"

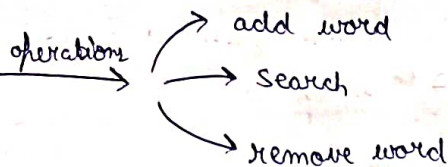
$n \rightarrow$ Total no. of words

$k \rightarrow$ word length

Hash code $\rightarrow O(k)$

$\left\{ \begin{array}{l} \text{if } n \gg k \approx \text{Hashcode} = O(1) \\ \text{No. of entries in a box} = n/k \\ \text{(Load factor)} \\ \text{we always ensure } n/k < 0.7 \end{array} \right.$

Trie



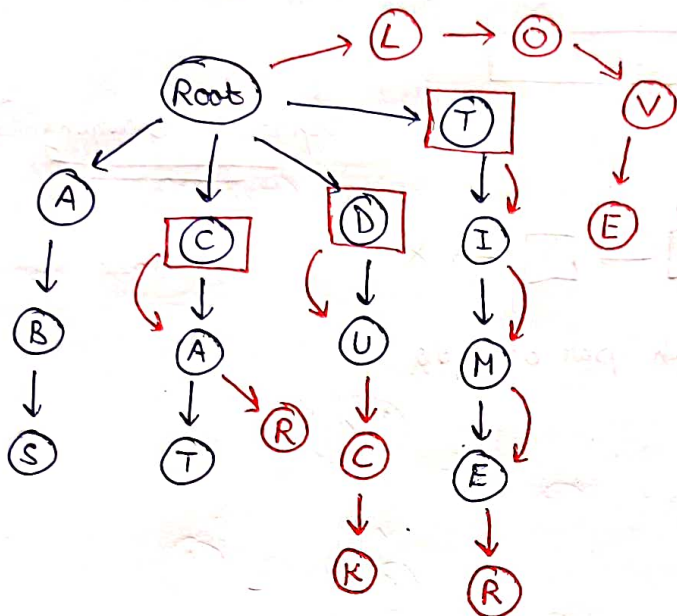
By Hashmap <string, bool>

$O(1) \Rightarrow$ avg. case

but in worst case T.C. differ

$O(l)$ $l = \text{length of word}$

Trie



① add word

Insert CAR

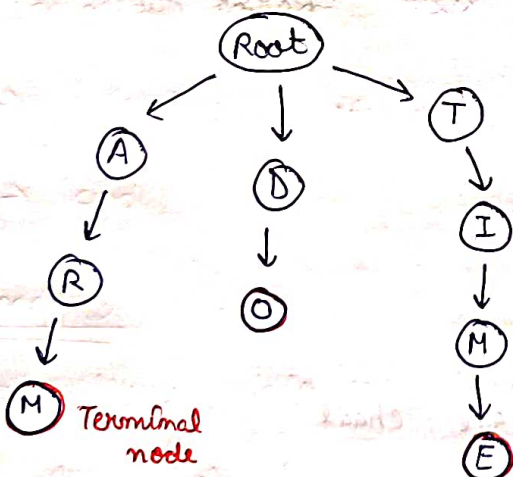
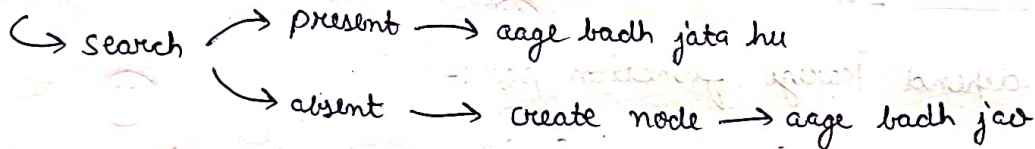
Insert DUCK

Insert TIMER

Insert LOVE

Algo

Character -



TrieNode {

char data;

TrieNode* children[26];

bool isTerminal;

}

Constructor

TrieNode(char ch) {

data = ch;

for (int i=0; i<26; i++) {

children[i] = NULL;

}

isTerminal = false;

}

```
class Trie {
```

```
public:
```

```
TrieNode* root;    }    Trie() {
```

```
    root = new TrieNode('');
```

```
void insertUtil(TrieNode* root, string word) {
```

```
    if (word.length() == 0) {
```

```
        root->isTerminal = true;
```

```
    }    return;
```

```
    int index = word[0] - 'A';
```

```
    TrieNode* child;
```

```
    if (root->children[index] != NULL) {
```

```
        child = root->children[index];
```

```
    }
```

```
    else {
```

```
        child = new TrieNode(word[0]);
```

```
        root->children[index] = child;
```

```
        insertUtil(child, word.substr(1));
```

```
    }
```

```
void insertWord(string word) {
```

```
    insertUtil(root, word);
```

```
}
```

```
bool searchUtil(TrieNode* root, string word) {
```

```
    if (word.length() == 0) {
```

```
        return root->isTerminal;
```

```
    }
```

```
    int index = word[0] - 'A';
```

```
    TrieNode* child;
```

```
    if (root->children[index] != NULL) {
```

```
        child = root->children[index];
```

```
    }
```

```
    else {
```

```
        return false;
```

```
return searchUtil(child, word.substr(1));
```

```
}
```

```
bool search(string word) {
```

```
    return searchUtil(root, word);
```

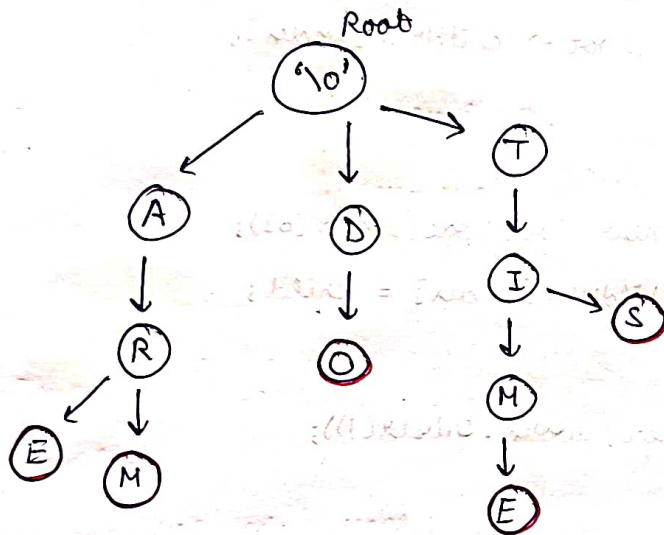
```
}
```

```
}
```

T.C. $\rightarrow$ Insertion $\Rightarrow O(L)$

$\rightarrow$ search $\Rightarrow O(L)$

Remove Word



Algo-

① search word

② isTerminal = false;

T.C. = $O(L)$

Qu- Longest Common Prefix

i/p $\Rightarrow$ { "coding", "codezen", "codingninja", "coders" }

o/p $\Rightarrow$ cod

Approach ①

First string pakdo aur character by character remaining me check karo

if ch match

$\rightarrow$ ans me add kardo

$\rightarrow$ aage badho

if ch ! match

$\rightarrow$ Ruk jao

String longestCommonPrefix (vector<string> &arr, int n) {

String ans = "";

for (int i=0; i<arr[0].length(); i++) {

char ch = arr[0][i];

bool match = true;

for (int j=1; j<n; j++) {

if (ch != arr[j][i] || arr[j].size < i) {

match = false;

break;

}

}

if (match == false) {

break;

}

else {

ans.push-back(ch);

}

return ans;

}

T.C. = $O(mn)$

m = length of 1st string

n = no of rem. strings

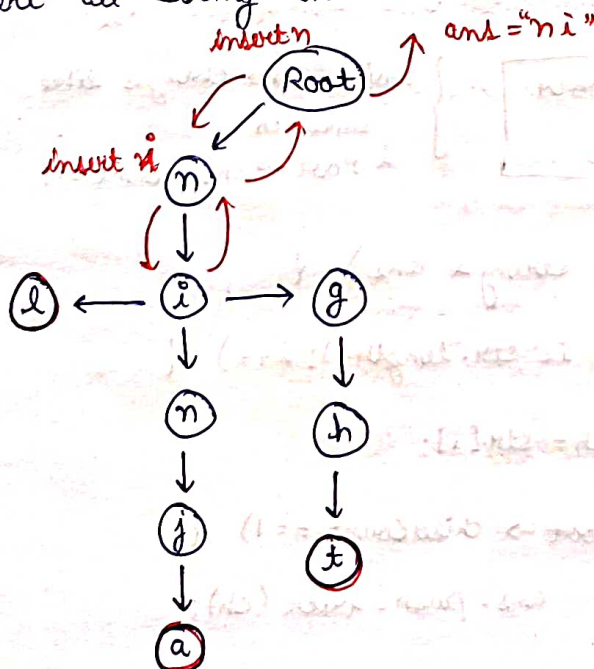
S.C. = $O(1)$

Approach 2

i/p $\Rightarrow$ ninja night nil

o/p $\Rightarrow$ ni

① Insert all string into Trie



if child count == 1

ans me add karo

else

return

```
class TrieNode {
```

```
public:
```

```
    char data;
```

```
    TrieNode* children[26];
```

```
    int childCount;
```

```
    bool isTerminal;
```

```
    TrieNode(char ch) {
```

```
        data = ch;
```

```
        for (int i=0; i<26; i++) {
```

```
            children[i] = NULL;
```

```
        }
```

```
        childCount = 0;
```

```
        isTerminal = false;
```

```
    }
```

```
}
```

```
class Trie {
```

```
public:
```

```
    TrieNode* root;
```

```
    Trie(char ch) {
```

```
        root = new TrieNode(ch);
```

```
    }
```

copy fn of insert word
from previous

} with some changes like
small ch
& root → childCount++;

```
void lcp(string str, string & ans) {
```

```
    for (int i=0; i<str.length(); i++) {
```

```
        char ch = str[i];
```

```
        if (root → childCount == 1) {
```

```
            ans.push_back(ch);
```

```
int index = ch - 'a';
```

```
root = root -> children[index];
```

```
}
```

```
else {
```

```
break;
```

```
}
```

```
if (root -> isTerminal) {
```

```
break;
```

```
}
```

```
}
```

```
}
```

```
};
```

```
string longestCommonPrefix(vector<string> &arr, int n) {
```

```
Tree * t = new Tree('\0');
```

```
for (int i=0; i<n; i++) {
```

```
t -> insertWord(arr[i]);
```

```
}
```

```
string first = arr[0];
```

```
string ans = "";
```

```
t -> lcp(first, ans);
```

```
return ans;
```

```
}
```

T.C. = $O(mn)$

S.C. = $O(mn)$

